

Cấu trúc tổng thể

- langgraph-labs/
 - |— README.md
 - |— pyproject.toml
 - |— .env.example
 - |
 - |— shared/
 - |— config.py
 - |— models.py
 - |— fake_models.py
 - |— fixtures.py
 - |— logging.py
 - |
 - |— labs/
 - |— 01_first_state_graph/
 - |— 02_state_and_reducers/
 - |— 03_edges_and_routing/
 - |— 04_cycles_and_limits/
 - |— 05_parallel_execution/
 - |— 06_send_map_reduce/
 - |— 07_command_control/
 - |— 08_functional_api/
 - |— 09_persistence_threads/
 - |— 10_interrupt_approval/
 - |— 11_time_travel/
 - |— 12_short_long_memory/
 - |— 13_streaming_events/
 - |— 14_fault_tolerance/
 - |— 15_subgraphs/
 - |— 16_tool_calling_agent/
 - |— 17_agentic_rag/
 - |— 18_multi_agent/
 - |— 19_production_graph/
 - |
 - |— projects/
 - |— agentic_operations_copilot/
 - |
 - |— tests/
 - |— shared/
 - |— projects/

Lab 01 — First StateGraph

- 01_first_state_graph/
 - |— README.md
 - |— state.py
 - |— nodes.py
 - |— graph.py
 - |— run.py
 - |— tests/
 - |— test_nodes.py
 - |— test_graph.py

Mục tiêu

Xây một workflow xử lý văn bản:

normalize → analyze → format

Người học phải hiểu:

- `TypedDict` state.
 - Partial state update.
 - `START`.
 - `END`.
 - `add_node`.
 - `add_edge`.
 - `compile`.
 - `invoke`.
 - Super-step tuyến tính.
-

Lab 02 — State và Reducers

- 02_state_and_reducers/
 - |— README.md
 - |— state.py
 - |— reducers.py
 - |— nodes.py
 - |— graph.py
 - |— experiments.py
 - |— tests/
 - |— test_reducers.py
 - |— test_overwrite.py
 - |— test_parallel_updates.py

Mục tiêu

So sánh:

- Default overwrite.
- `operator.add`.
- Custom reducer.
- `add_messages`.
- Conflict khi parallel write.
- Reducer deterministic.
- Reducer cho dictionary và ranked results.

Bài lab cố tình tạo lỗi concurrent state update, sau đó sửa bằng reducer phù hợp.

Lab 03 — Edges và Conditional Routing

- 03_edges_and_routing/
 - |— README.md
 - |— state.py
 - |— nodes/
 - |— classify.py
 - |— process_text.py
 - |— process_number.py
 - |— reject.py
 - |— routers.py
 - |— graph.py
 - |— run_cases.py
 - |— tests/
 - |— test_router.py
 - |— test_paths.py

Mục tiêu

Xây input processor có ba nhánh:

- text
- number

unsupported

Nội dung:

- Static edges.
- Conditional edges.
- Conditional entry point.
- Typed route bằng `Literal`.
- Router deterministic.

- Test từng graph path.
-

Lab 04 — Cycles và Termination

- 04_cycles_and_limits/
 - |— README.md
 - |— state.py
 - |— nodes/
 - |— generate.py
 - |— evaluate.py
 - |— revise.py
 - |— routers.py
 - |— graph.py
 - |— run.py
 - |— tests/
 - |— test_success_stop.py
 - |— test_max_attempts.py
 - |— test_recursion_limit.py

Mục tiêu

Xây evaluator-optimizer:

- generate → evaluate
- |— pass → END
 - |— fail → revise → evaluate

Phân biệt:

- Business stop condition.
 - `max_attempts`.
 - Recursion limit.
 - Success threshold.
 - Stop reason.
 - Infinite loop prevention.
-

Lab 05 — Parallel Execution và Reducer Merge

- 05_parallel_execution/
 - |— README.md
 - |— state.py
 - |— nodes/

- | — financial_analysis.py
- | — risk_analysis.py
- | — technical_analysis.py
- | — synthesize.py
- | — reducers.py
- | — graph.py
- | — benchmarks.py
- | — tests/
- | — test_parallel_merge.py
- | — test_latency.py

Mục tiêu

Ba node phân tích độc lập chạy song song rồi merge.

Người học quan sát:

- Super-step.
 - Parallel nodes.
 - Reducer.
 - Fan-in.
 - Latency tuần tự so với song song.
 - Shared state snapshot.
-

Lab 06 — Send và Dynamic Map-Reduce

- 06_send_map_reduce/
- | — README.md
- | — state.py
- | — worker_state.py
- | — nodes/
- | | — split_document.py
- | | — analyze_chunk.py
- | | — aggregate.py
- | — dispatchers.py
- | — reducers.py
- | — graph.py
- | — sample_data/
- | — tests/
- | — test_dynamic_fanout.py
- | — test_worker_state.py
- | — test_reduce.py

Mục tiêu

Phân tích tài liệu có số chunk động:

- split
- ↓
- Send(analyze_chunk × n)
- ↓

aggregate

Nội dung:

- Dynamic fan-out.
 - Worker-local input.
 - Global reducer.
 - Bounded concurrency.
 - Partial worker failure.
 - Ordering và deduplication.
-

Lab 07 — **Command** và Unified Control Flow

- 07_command_control/
 - |— README.md
 - |— state.py
 - |— nodes/
 - |— inspect_request.py
 - |— safe_handler.py
 - |— risky_handler.py
 - |— finish.py
 - |— graph.py
 - |— tests/
 - |— test_command_update.py
 - |— test_command_goto.py
 - |— test_control_paths.py

Mục tiêu

Thay conditional edge bằng **Command** trong các quyết định cần đồng thời:

- Update state.
- Route node.
- Ghi lý do routing.
- Chuyển sang fallback.

So sánh ưu, nhược điểm của router riêng và **Command**.

Lab 08 — Functional API

- 08_functional_api/
 - |— README.md
 - |— tasks.py
 - |— workflow.py
 - |— run.py
 - |— replay_experiment.py
 - |— tests/
 - |— test_tasks.py
 - |— test_entrypoint.py
 - |— test_replay.py

Mục tiêu

Chuyển một workflow Python procedural sang:

- `@task`.
- `@entrypoint`.
- Task futures.
- Async task.
- Persistence.
- Replay.
- Idempotency.
- Non-determinism boundary.

Thực hiện cùng bài toán bằng Graph API để so sánh.

Lab 09 — Persistence, Threads và Checkpoints

- 09_persistence_threads/
 - |— README.md
 - |— state.py
 - |— nodes.py
 - |— graph.py
 - |— persistence/
 - |— memory.py
 - |— sqlite.py
 - |— inspect_state.py
 - |— inspect_history.py
 - |— tests/
 - |— test_thread_isolation.py

- |— test_resume.py
- |— test_history.py

Mục tiêu

Xây conversation workflow có:

- Nhiều thread.
 - Nhiều run trên một thread.
 - `get_state`.
 - `get_state_history`.
 - Checkpoint metadata.
 - Thread isolation.
 - Process restart với persistent backend.
-

Lab 10 — Human Approval bằng Interrupt

- 10_interrupt_approval/
- |— README.md
- |— state.py
- |— nodes/
- | |— prepare_action.py
- | |— request_approval.py
- | |— execute_action.py
- | |— reject_action.py
- |— routers.py
- |— graph.py
- |— cli_approval.py
- |— tests/
- | |— test_interrupt.py
- | |— test_approve.py
- | |— test_reject.py
- |— test_idempotency.py

Mục tiêu

Agent chuẩn bị thao tác gửi email hoặc cập nhật database, nhưng phải chờ phê duyệt.

Nội dung:

- Dynamic interrupt.
- Interrupt payload.
- `Command(resume=...)`.
- Approve, reject, edit.

- Node replay.
 - Side effect placement.
 - Duplicate resume protection.
-

Lab 11 — Time Travel và Fork

- 11_time_travel/
 - |— README.md
 - |— state.py
 - |— nodes.py
 - |— graph.py
 - |— replay.py
 - |— fork.py
 - |— compare_branches.py
 - |— tests/
 - |— test_replay.py
 - |— test_fork.py
 - |— test_original_history.py

Mục tiêu

Chạy report workflow, sau đó:

- Chọn checkpoint cũ.
 - Replay.
 - Sửa state.
 - Fork nhánh mới.
 - So sánh output hai nhánh.
 - Chứng minh lịch sử gốc không bị xóa.
-

Lab 12 — Short-term và Long-term Memory

- 12_short_long_memory/
 - |— README.md
 - |— state.py
 - |— context.py
 - |— graph.py
 - |— nodes/
 - |— chat.py
 - |— extract_memory.py
 - |— retrieve_memory.py
 - |— summarize_history.py

- | └─ trim_messages.py
- |── memory/
- | └─ namespaces.py
- | └─ store.py
- | └─ policies.py
- |── tests/
- | └─ test_thread_memory.py
- | └─ test_cross_thread_memory.py
- | └─ test_namespace_isolation.py
- └─ test_memory_policy.py

Mục tiêu

Xây assistant nhớ preference xuyên thread.

Nội dung:

- Short-term message state.
 - Long-term Store.
 - Namespace.
 - Runtime context.
 - Memory extraction.
 - Semantic search.
 - Trimming.
 - Summarization.
 - Privacy và expiration.
-

Lab 13 — Streaming Events

- 13_streaming_events/
- |── README.md
- |── state.py
- |── nodes.py
- |── graph.py
- |── stream_console.py
- |── event_consumer.py
- |── custom_channels.py
- |── tests/
- | └─ test_message_stream.py
- | └─ test_update_stream.py
- └─ test_custom_progress.py

Mục tiêu

Xây research workflow phát:

- Model tokens.
- Tool lifecycle.
- Node updates.
- Progress.
- Subgraph events.
- Final output.

So sánh:

- `updates`.
 - `values`.
 - `messages`.
 - Event streaming v3.
 - Custom stream channel.
-

Lab 14 — Fault Tolerance

- 14_fault_tolerance/
 - |— README.md
 - |— state.py
 - |— graph.py
 - |— nodes/
 - |— unstable_api.py
 - |— long_running.py
 - |— payment.py
 - |— compensate.py
 - |— fallback.py
 - |— policies/
 - |— retries.py
 - |— timeouts.py
 - |— error_handlers.py
 - |— tests/
 - |— test_retry_success.py
 - |— test_retry_exhausted.py
 - |— test_timeout.py
 - |— test_compensation.py
 - |— test_partial_parallel_failure.py

Mục tiêu

Mô phỏng:

- Network failure.

- Rate limit.
- Async node tree.
- Parallel branch lỗi.
- Payment failure.
- Saga compensation.
- Resume sau crash.

Nội dung mới của LangGraph 1.2:

- Node timeout.
- Idle heartbeat.
- Node error handler.
- Graph-wide defaults.
- Recovery bằng **Command**.

Lab 15 — Subgraphs và Modular Workflow

- 15_subgraphs/
 - |— README.md
 - |— parent/
 - |— state.py
 - |— graph.py
 - |— nodes.py
 - |— subgraphs/
 - |— research/
 - |— state.py
 - |— graph.py
 - |— nodes.py
 - |— writing/
 - |— state.py
 - |— graph.py
 - |— nodes.py
 - |— adapters/
 - |— research_adapter.py
 - |— writing_adapter.py
 - |— tests/
 - |— test_shared_schema.py
 - |— test_private_schema.py
 - |— test_subgraph_persistence.py

Mục tiêu

Thực hành:

- Add compiled subgraph trực tiếp.

- Wrapper adapter khi schema khác nhau.
 - Private subgraph state.
 - Per-invocation persistence.
 - Per-thread persistence.
 - Stream subgraph output.
 - Inspect nested state.
-

Lab 16 — Tool-calling Agent

- 16_tool_calling_agent/
 - |— README.md
 - |— state.py
 - |— context.py
 - |— graph.py
 - |— nodes/
 - |— model_node.py
 - |— policy_node.py
 - |— finalizer.py
 - |— tools/
 - |— calculator.py
 - |— inventory.py
 - |— customer_orders.py
 - |— schemas.py
 - |— policies/
 - |— tool_permissions.py
 - |— budgets.py
 - |— argument_validation.py
 - |— tests/
 - |— test_single_tool.py
 - |— test_multiple_tools.py
 - |— test_tool_loop.py
 - |— test_tool_budget.py
 - |— test_permission_boundary.py

Mục tiêu

Xây agent phải gọi nhiều tool để giải một query.

Nội dung:

- Model binding.
- **ToolNode**.
- Tool loop.
- Parallel tool calls.
- Multiple tool rounds.

- ToolRuntime.
 - Inject trusted user context.
 - Tool budget.
 - Tool policy.
 - Tool argument validation.
 - Model không được tự cung cấp identity.
-

Lab 17 — Agentic RAG

- 17_agentic_rag/
 - |— README.md
 - |— state.py
 - |— graph.py
 - |— nodes/
 - |— analyze_query.py
 - |— decide_retrieval.py
 - |— retrieve.py
 - |— rerank.py
 - |— grade_documents.py
 - |— rewrite_query.py
 - |— generate.py
 - |— check_groundedness.py
 - |— check_answer_quality.py
 - |— fallback.py
 - |— routers/
 - |— retrieval_router.py
 - |— evidence_router.py
 - |— answer_router.py
 - |— schemas/
 - |— relevance_grade.py
 - |— groundedness_grade.py
 - |— answer_grade.py
 - |— retrieval/
 - |— vector_store.py
 - |— keyword_search.py
 - |— hybrid_search.py
 - |— tests/
 - |— test_no_retrieval_path.py
 - |— test_rewrite_path.py
 - |— test_grounded_answer.py
 - |— test_max_attempts.py
 - |— evaluation_dataset.json

Mục tiêu

Xây RAG có khả năng:

- Quyết định có retrieve không.
 - Hybrid retrieval.
 - Rerank.
 - Grade document.
 - Rewrite query.
 - Generate.
 - Kiểm tra groundedness.
 - Kiểm tra usefulness.
 - Fallback khi không đủ evidence.
 - Chặn loop bằng budget.
-

Lab 18 — Multi-agent System

- 18_multi_agent/
 - |— README.md
 - |— parent_state.py
 - |— supervisor_graph.py
 - |— routing.py
 - |— handoff.py
 - |— agents/
 - |— research_agent/
 - |— state.py
 - |— graph.py
 - |— tools.py
 - |— data_agent/
 - |— state.py
 - |— graph.py
 - |— tools.py
 - |— report_agent/
 - |— state.py
 - |— graph.py
 - |— tools.py
 - |— context/
 - |— context_builder.py
 - |— message_filter.py
 - |— summary.py
 - |— tests/
 - |— test_routing.py
 - |— test_handoff.py
 - |— test_private_context.py
 - |— test_parallel_agents.py
 - |— test_termination.py

Mục tiêu

So sánh ba kiến trúc:

1. Router → specialist.
2. Supervisor → nhiều specialist.
3. Agent-to-agent handoff.

Tập trung vào:

- Subgraph.
 - Private state.
 - Shared output contract.
 - Active agent.
 - Handoff history.
 - Context engineering.
 - Loop prevention.
 - Specialist tool boundary.
-

Lab 19 — Production Graph

- 19_production_graph/
 - |— README.md
 - |— pyproject.toml
 - |— langgraph.json
 - |— .env.example
 - |— src/
 - |— production_agent/
 - |— graph.py
 - |— state.py
 - |— context.py
 - |— nodes/
 - |— routers/
 - |— tools/
 - |— subgraphs/
 - |— persistence/
 - |— policies/
 - |— telemetry/
 - |— scripts/
 - |— seed_data.py
 - |— inspect_threads.py
 - |— migrate_state.py
 - |— tests/
 - |— unit/
 - |— integration/
 - |— reliability/

- └─ migration/
 - └─ evaluation/

Mục tiêu

Đưa graph từ notebook thành application:

- Dependency management.
 - `langgraph.json`.
 - Local Agent Server.
 - Studio.
 - Persistent checkpointing.
 - Long-term store.
 - Structured logging.
 - Trace.
 - Environment configuration.
 - State migration.
 - Evaluation.
 - Load test.
 - Failure injection.
 - Security boundary.
-

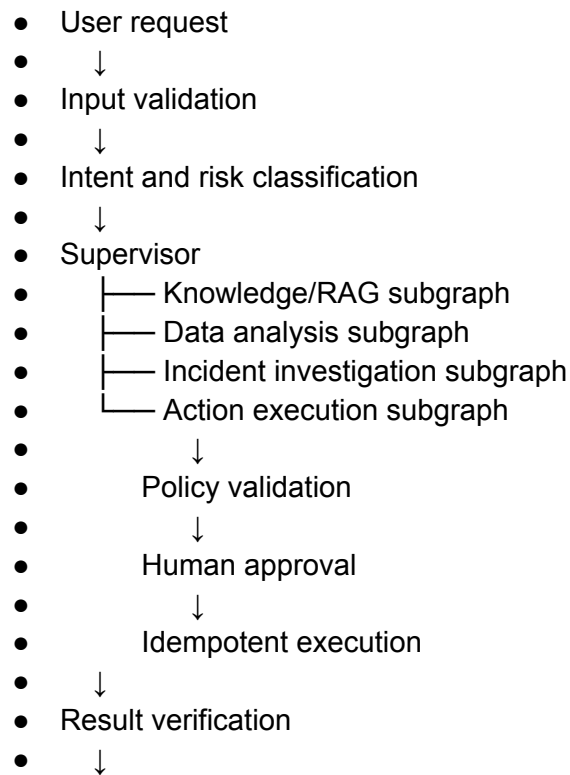
Phần XXXIV — Project tổng hợp: Agentic Operations Copilot

132. Bài toán

Xây assistant hỗ trợ vận hành doanh nghiệp có thể:

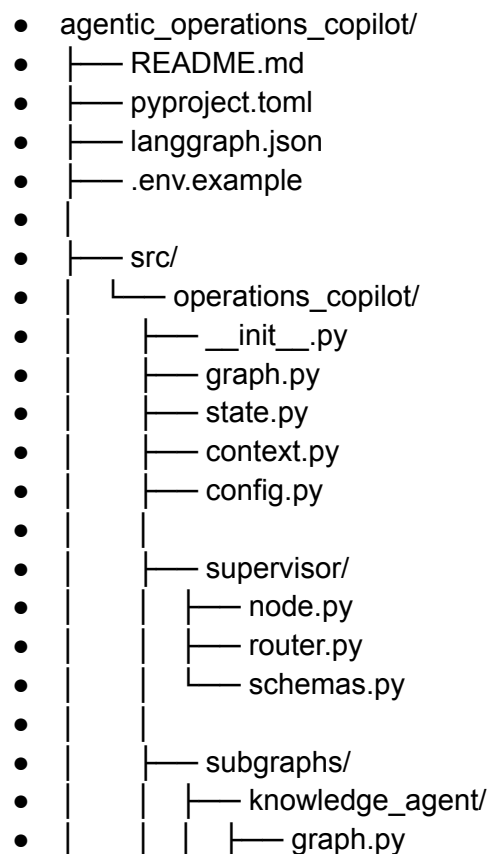
- Trả lời câu hỏi dựa trên tài liệu.
- Tra cứu trạng thái đơn hàng.
- Phân tích sự cố.
- Viết kế hoạch hành động.
- Yêu cầu phê duyệt trước thao tác thay đổi dữ liệu.
- Giao nhiệm vụ cho specialist agents.
- Nhớ preference người dùng.
- Khôi phục khi API lỗi.
- Stream tiến trình lên UI.
- Audit toàn bộ quyết định.

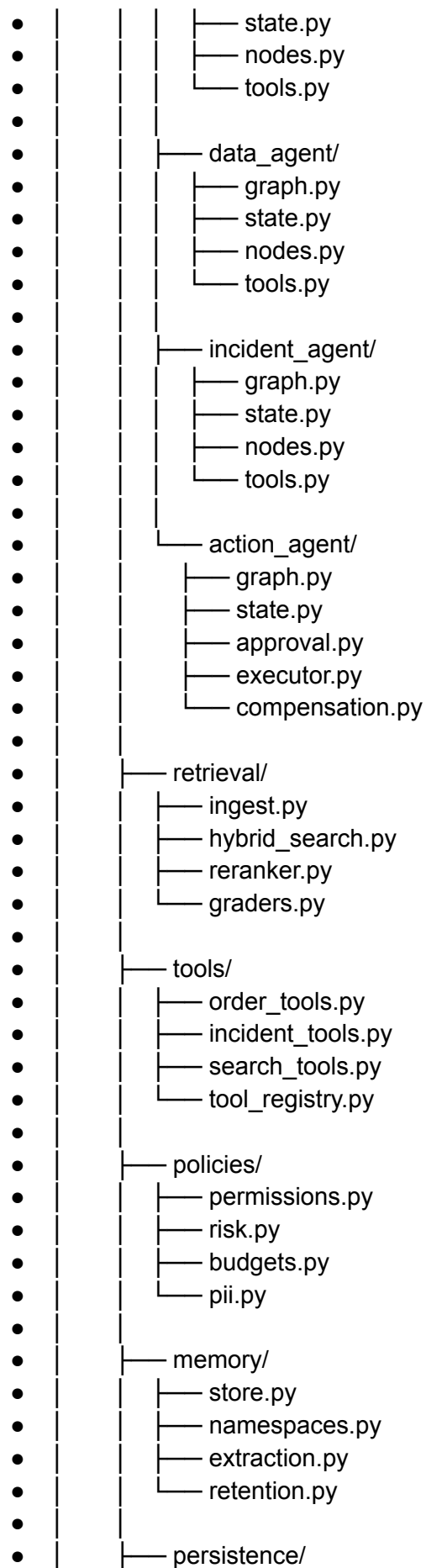
133. Kiến trúc



Final response

134. Cấu trúc source





- | | — test_duplicate_execution.py
- | | — test_compensation.py
- |
- | — migration/
- | | — test_old_checkpoints.py
- |
- | — evaluation/
- | | — dataset.json
- | | — test_tool_selection.py
- | | — test_groundedness.py
- |
- — test_task_success.py

135. Những kiến thức được tích hợp

Project tổng hợp bắt buộc sử dụng:

- StateGraph.
 - Typed state.
 - Reducer.
 - Static và conditional edges.
 - Loop.
 - Command.
 - Send.
 - Subgraph.
 - ToolNode hoặc custom tool execution.
 - Runtime context.
 - Checkpointer.
 - Store.
 - Interrupt.
 - Resume.
 - Time travel.
 - Streaming.
 - Retry.
 - Timeout.
 - Error handler.
 - Idempotency.
 - Saga compensation.
 - State migration.
 - Unit test và graph-path test.
 - Observability.
 - Agentic RAG.
 - Supervisor hoặc handoff multi-agent.
-

Thứ tự học khuyến nghị

Không nên học multi-agent ngay từ đầu. Thứ tự hiệu quả là:

- StateGraph cơ bản
- ↓
- State và reducer
- ↓
- Edge, routing và loop
- ↓
- Parallel execution
- ↓
- Send và Command
- ↓
- Functional API
- ↓
- Persistence và checkpoint
- ↓
- Interrupt và time travel
- ↓
- Memory và streaming
- ↓
- Retry, timeout, error handling
- ↓
- Subgraph
- ↓
- Tool-calling agent
- ↓
- Agentic RAG
- ↓
- Multi-agent
- ↓

Production engineering

Trọng tâm cốt lõi của LangGraph không phải là ghi nhiều node. Kỹ năng quan trọng nhất là thiết kế đúng:

- State contract
- + transition logic
- + persistence boundary
- + failure boundary
- + side-effect boundary
-
- + human-control boundary

Khi sáu ranh giới này rõ ràng, graph thường dễ kiểm thử, dễ khôi phục và đủ an toàn để tiến từ demo sang hệ thống agentic thực tế.

